

Game

Animation

How animation works



Animation = Sprite + Events

Sprite

2D static image in games

Animation

modify a sprite each game update (Event::execute)

Sequence of events

- We can use the Event system to animate attacks
- Without animation
 - `Spear::use` → Hit Event
- With animation
 - `Spear::use` → Thrust Event
 - Thrust Event → Hit Event

Event interface

To animate, we consider 3 member functions

```
// engine/event.h
class Event {
public:
    // animation duration (10 frames = 1 second)
    Event(int number_of_frames=1);

    // called every 0.1 seconds, modify sprite in here
    virtual void execute(Engine& engine) = 0;

    // generate Hit event when animation is done
    virtual void when_done(Engine& engine);
};
```

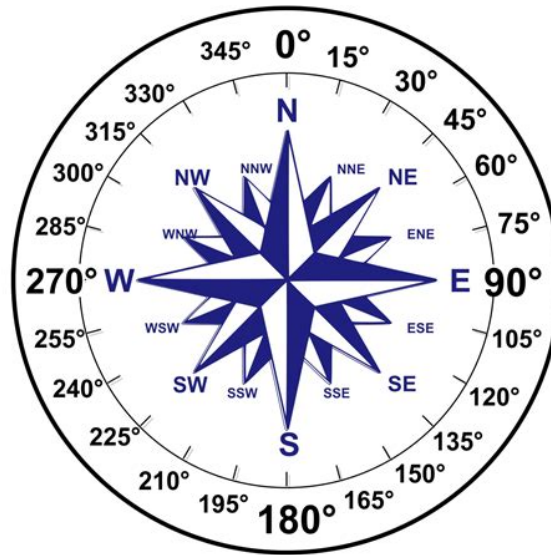
class Sprite

```
// engine/graphics/sprite.h
struct Sprite {
    int texture_id{-1}; // internal to graphics engine
    Vec location{0, 0}; // upper left corner of sprite in image
    Vec size{0, 0};      // size (width, height) of sprite in image
    Vec shift{0, 0};     // pixels to shift by when displaying
    Vec center{0, 0};    // position to rotate about
    double angle{0.0};   // angle of rotation
    bool flip{false};    // whether to flip sprite horizontally
};
```

class Sprite

For most animation, we only need to consider

```
// represents a 2D image in the game
struct Sprite {
    Vec shift{0, 0};    // pixels to shift by when displaying
    double angle{0.0};  // angle of rotation
};
```



Thrust Item Event

Remember that class Item has a sprite

```
// engine/item.h
class Item {
public:
    explicit Item(std::string name);

    std::string name;
    Sprite sprite;
};
```

```
class Thrust : public Event {
public:
    Thrust(Sprite& sprite, Vec direction);
    // weapon sprite^ entity facing^

private:
    Sprite& sprite; // item sprite, will be modified
    Sprite copy;
    Vec direction;
};
```

Thrust Item Event

Connecting Item's sprite and Thrust Event

```
void Spear::use(Engine& engine, Entity& attacker, Entity& defender) {  
    Vec direction = defender.get_position() - attacker.get_position();  
    auto thrust = engine.events.create_event<Thrust>(sprite, direction);  
    thrust->add_next<Hit>(defender, damage);  
}
```

Thrust Item Event

To set animation duration, pass the number of frames to the Event constructor

```
constexpr int duration = 3; // number of frames
constexpr int delta = 3;    // shift per frame in pixels

Thrust::Thrust(Sprite& sprite, Vec direction)
    :Event{duration}, sprite{sprite}, copy{sprite}, direction{direction} {
    // orient spear along direction
    // shift spear to make it look nice
}
```

Thrust Item Event

To animate the sprite, override execute

```
void Thrust::execute(Engine&) {  
    // shift in pixels  
    sprite.shift += direction * delta;  
}
```

Resetting the sprite

- During animation, you are modifying the original Item sprite
- **You must set it back to the original when done!**

```
class Thrust : public Event {
private:
    Sprite& sprite; // modified during animation
    Sprite copy;    // copy before animation
};

Thrust::Thrust(Sprite& sprite, Vec direction)
    :Event{duration}, sprite{sprite}, copy{sprite},
    // ^ save a copy of original

void Thrust::when_done(Engine&) {
    sprite = copy; // restore to original
}
```

Demo

Swing event (translation + rotation)

Demo

Animation event at position

Fire Event

To place fire on a dungeon tile, we need four parts:

- An animation event
- Load the "fire" sprite at the start of the Event
- Understand AnimatedSprite
- Use Camera to add an overlay (sprite on top of most other sprites)

Fire Event

```
class Animation : public Event {
public:
    Animation(const Vec& position, const std::string& sprite_name

    // called when the event is first run
    void on_start(Engine& engine) override;
    void execute(Engine& engine) override;

private:
    Vec position;
    std::string sprite_name;
    AnimatedSprite sprite;
};
```

Fire Event

The constructor has only the sprite name, load it upon starting

```
void Animation::on_start(Engine& engine) {  
    sprite = engine.graphics.get_animated_sprite(sprite_name);  
    number_of_frames = sprite.number_of_frames();  
}
```

```
class AnimatedSprite {  
public:  
    AnimatedSprite();  
    AnimatedSprite(const std::vector<Sprite>& sprites,  
                  int ticks_per_frame, int starting_frame=0);  
    void flip(bool flip); // flip sprite horizontally  
    void update();        // move to next frame of animation  
    Sprite get_sprite() const;  
    int number_of_frames() const;  
    bool visible;  
  
private:  
    std::vector<Sprite> sprites;  
    int ticks_per_frame{1}, current_frame{0}, time{0};  
};
```


Fire Event

```
void Animation::execute(Engine& engine) {  
    engine.camera.add_overlay(position, sprite.get_sprite());  
    sprite.update();  
}
```

Sequencing animations and hits

Sequence all events when creating the first

```
Raise -> Fire -> Hit  
      -> Audio
```

```
void Staff::use(Engine& engine, Entity&, Entity& defender) {  
    auto raise = engine.events.create_event<Raise>(sprite);  
    auto fire = std::make_shared<Animation>(defender.get_position(), "fire");  
    fire->add_next<Hit>(defender, damage);  
    raise->add_next(fire);  
    raise->add_next<AudioEvent>("fire");  
}
```

Sound!

```
class AudioEvent : public Event {  
public:  
    explicit AudioEvent(const std::string& name);  
    void execute(Engine& engine) override;  
  
private:  
    std::string name;  
};
```

```
void AudioEvent::execute(Engine& engine) {  
    engine.audio.play_sound(name);  
}
```

Checkpoint 4

- Define **one** new animation
- Update your weapons to generate animation events
- When animation events are done, generate Hit events
- Examples:
 - **Raise**: raise your weapon in the air
 - **Spin Attack**: rotate weapon around hero
 - **Throw**: translation + rotation